

Ausführliche Beweise für Model Checking × Roboternavigation

Stephan Epp

24. Juni 2026

Inhaltsverzeichnis

1	TikZ-Diagramm: Pattern für Annäherung an Ziel	2
2	Ausführliche Beweise für Kapitel 3: Verifikation mit dem Subgraph Algorithmus	3
2.1	Beweis: Korrektheit der Verifikation	3
2.2	Beweis: Laufzeit der Verifikation	5
3	Ausführliche Beweise für Kapitel 4: Theoretische Garantien der Phi-Navigation	8
3.1	Beweis: Garantierte Zielerreichung	8
3.2	Beweis: Minimale Energiedissipation	11
3.3	Beweis: Robustheit gegenüber Umgebungsänderungen	15

1 TikZ-Diagramm: Pattern für Annäherung an Ziel

2 Ausführliche Beweise für Kapitel 3: Verifikation mit dem Subgraph Algorithmus

2.1 Beweis: Korrektheit der Verifikation

Theorem 2.1 (Korrektheit der Verifikation). Der Algorithmus `Verify Navigation Property` liefert genau dann `True`, wenn das Property Pattern G_P als Subgraph in der Navigations-Kripke-Struktur $\mathcal{K}_\phi$ existiert.

Beweis. Wir zeigen die Äquivalenz in beiden Richtungen durch einen konstruktiven Beweis, der auf der Injektivität der Signatur-Funktion und der Vollständigkeit der zyklischen Rotationen basiert.

Teil 1: Hinrichtung ($\Rightarrow$): Wenn der Algorithmus `True` zurückgibt, dann existiert G_P als Subgraph in $\mathcal{K}_\phi$.

Angenommen, der Algorithmus gibt `True` zurück. Dies geschieht genau dann in Zeile 9 des Algorithmus, wenn für eine bestimmte Rotation k die Bedingung in Zeile 8 erfüllt ist: Das rotierte Signatur-Array von G_P ist als Subsequenz im Signatur-Array von $\mathcal{K}_\phi$ enthalten.

Sei $\text{Sig}_A = [\sigma_0^A, \sigma_1^A, \dots, \sigma_{n_A-1}^A]$ das Signatur-Array der Kripke-Struktur $\mathcal{K}_\phi$ mit Adjazenzmatrix A der Größe $n_A \times n_A$, und sei $\text{Sig}_B = [\sigma_0^B, \sigma_1^B, \dots, \sigma_{n_B-1}^B]$ das Signatur-Array des Property Pattern Graphs G_P mit Adjazenzmatrix B der Größe $n_B \times n_B$.

Die Signatur für Spalte j ist definiert als:

$$\sigma_j = \sum_{i=0}^{n-1} M_{ij} \cdot 2^i + j \cdot 2^n \quad (1)$$

wobei M die entsprechende Adjazenzmatrix ist. Diese Funktion kodiert sowohl den Spaltenvektor (durch die Summe $\sum_{i=0}^{n-1} M_{ij} \cdot 2^i$, die eine binäre Darstellung darstellt) als auch die Spaltenposition (durch den Term $j \cdot 2^n$).

Wenn die rotierte Sequenz $\text{Sig}_B^{(k)} = [\sigma_k^B, \sigma_{k+1}^B, \dots, \sigma_{n_B-1}^B, \sigma_0^B, \dots, \sigma_{k-1}^B]$ als Subsequenz in Sig_A an Position i_0 gefunden wird, bedeutet dies:

$$\sigma_{i_0+j}^A = \sigma_{(k+j)}^B \mod n_B \quad \text{für alle } j \in \{0, 1, \dots, n_B - 1\} \quad (2)$$

Nach dem Lemma über die Eindeutigkeit der Signaturen (das zeigt, dass die Signatur-Funktion injektiv ist) impliziert die Gleichheit der Signaturen die Gleichheit der zugrundeliegenden Spaltenvektoren. Das heißt, wenn wir die Spaltengewichtung entfernen (durch Modulo 2^n Operation), erhalten wir:

$$\sigma_{i_0+j}^A \mod 2^{n_A} = \sigma_{(k+j)}^B \mod n_B \mod 2^{n_B} \quad (3)$$

Dies bedeutet, dass die Spaltenvektoren von B (nach zyklischer Rotation um k Positionen) mit einer zusammenhängenden Sequenz von Spaltenvektoren in A übereinstimmen. Da die Adjazenzmatrix die vollständige Graphstruktur kodiert – Spalte j repräsentiert alle eingehenden Kanten zu Knoten j –, bedeutet die Übereinstimmung der Spaltenvektoren, dass die lokale Nachbarschaftsstruktur identisch ist.

Konkret existiert eine Einbettung $\phi : V_B \rightarrow V_A$ (wobei V_B die Knotenmenge von G_P und V_A die Knotenmenge von $\mathcal{K}_\phi$ ist) mit:

$$\phi(v_j) = v_{i_0 + ((j-k) \bmod n_B)} \quad (4)$$

die die Graphstruktur erhält: Wenn $(v_i, v_j) \in E_B$ (Kante in G_P), dann ist $(\phi(v_i), \phi(v_j)) \in E_A$ (entsprechende Kante in $\mathcal{K}_\phi$). Dies folgt direkt aus der Tatsache, dass $B_{ij} = 1$ genau dann, wenn $A_{\phi(i), \phi(j)} = 1$, was durch die Übereinstimmung der Spaltenvektoren garantiert ist.

Somit ist G_P tatsächlich als Subgraph in $\mathcal{K}_\phi$ enthalten.

Teil 2: Rückrichtung ($\Leftarrow$): Wenn G_P als Subgraph in $\mathcal{K}_\phi$ existiert, dann gibt der Algorithmus *True* zurück.

Angenommen, G_P existiert als Subgraph in $\mathcal{K}_\phi$. Das bedeutet, es existiert eine injektive Funktion $\phi : V_B \rightarrow V_A$, die die Graphstruktur erhält: Für alle Kanten $(v_i, v_j) \in E_B$ gilt $(\phi(v_i), \phi(v_j)) \in E_A$.

Ohne Beschränkung der Allgemeinheit können wir annehmen, dass die Knoten von G_P so nummeriert sind, dass $\phi(v_j) = v_{i_0 + j}$ für ein gewisses $i_0 \in \{0, 1, \dots, n_A - n_B\}$ gilt. Falls dies nicht der Fall ist, können wir die Knoten von G_P umnummerieren, was einer zyklischen Rotation der Adjazenzmatrix B entspricht – genau dies wird durch die Schleife in Zeile 6 des Algorithmus abgedeckt.

Für diese Einbettung gilt: Die Spalte j von B entspricht der Spalte $i_0 + j$ von A , das heißt:

$$B_{ij} = A_{(i_0+i)(i_0+j)} \quad \text{für alle } i, j \in \{0, \dots, n_B - 1\} \quad (5)$$

Nach Definition der Signatur-Funktion haben wir dann für die Zeilenkomponenten (ohne Spaltengewichtung):

$$\sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i = \sum_{i=0}^{n_B-1} A_{(i_0+i)(i_0+j)} \cdot 2^i \quad (6)$$

$$= \text{Subsequenz der Zeilenkomponente von } \sigma_{i_0+j}^A \quad (7)$$

Die Spaltengewichtung unterscheidet sich zwar ($j \cdot 2^{n_B}$ vs. $(i_0 + j) \cdot 2^{n_A}$), aber diese wird in Zeile 8 des Algorithmus durch Extraktion der Zeilenkomponenten (Modulo-Operation) entfernt.

Somit wird bei der Rotation $k = 0$ (oder einer geeigneten Rotation, falls die Nummerierung anders ist) das rotierte Signatur-Array von G_P als Subsequenz in Sig_A gefunden, und der Algorithmus gibt **True** in Zeile 9 zurück.

Schlussfolgerung: Wir haben beide Richtungen der Äquivalenz gezeigt. Der Algorithmus gibt genau dann **True** zurück, wenn G_P als Subgraph in $\mathcal{K}_\phi$ existiert. Dies beweist die Korrektheit des Verifikationsalgorithmus. $\square$

2.2 Beweis: Laufzeit der Verifikation

Theorem 2.2 (Laufzeit der Verifikation). Die Verifikation eines Property Patterns mit m Zuständen in einer Navigations-Kripke-Struktur mit n Zuständen hat Laufzeit:

$$T(n, m) = O(n^2 + m \cdot n) = O(n^2)$$

für $m \ll n$.

Beweis. Wir analysieren die Laufzeit des Algorithmus **Verify Navigation Property** Zeile für Zeile und summieren die Komplexitäten der einzelnen Schritte.

Schritt 1: Konvertierung zu Adjazenzmatrizen (Zeilen 4-5)

Die Kripke-Struktur $\mathcal{K}_\phi$ wird zur Adjazenzmatrix A konvertiert. Angenommen, $\mathcal{K}_\phi$ ist als Liste von Zuständen mit Transitionsrelation gegeben. Die Konstruktion der $n \times n$ Adjazenzmatrix erfordert das Durchlaufen aller Zustandspaare und Prüfen, ob eine Transition existiert. Im schlechtesten Fall (vollständig verbundener Graph) hat dies Laufzeit $O(n^2)$. In der Praxis ist die Kripke-Struktur oft dünn besetzt (sparse), sodass die Laufzeit $O(n + |E|)$ sein kann, wobei $|E|$ die Anzahl der Kanten ist. Für die obere Schranke nehmen wir $O(n^2)$ an.

Die Konvertierung von G_P zur Matrix B hat analog Laufzeit $O(m^2)$.

Gesamtkomplexität für Schritt 1: $O(n^2 + m^2)$.

Schritt 2: Signaturberechnung für $\mathcal{K}_\phi$ (Zeilen 7-9)

Für jede Spalte $j \in \{0, \dots, n-1\}$ der Matrix A wird die Signatur berechnet:

$$\sigma_j^A = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n \quad (8)$$

Die Berechnung der Summe $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ erfordert n Multiplikationen und $n-1$ Additionen, also $O(n)$ Operationen pro Spalte. Da wir dies für alle n Spalten durchführen, ergibt sich eine Gesamtkomplexität von $O(n \cdot n) = O(n^2)$.

Es ist wichtig zu beachten, dass die Potenzoperationen 2^i effizient berechnet werden können: Entweder durch Bit-Shift-Operationen (die in konstanter Zeit laufen auf modernen Prozessoren) oder durch iteratives Verdoppeln. Selbst wenn wir konservativ annehmen, dass jede Potenzierung $O(\log n)$ Zeit benötigt (für große n), ist die Gesamtkomplexität $O(n^2 \log n)$, was asymptotisch immer noch polynomiell ist.

Gesamtkomplexität für Schritt 2: $O(n^2)$ (oder $O(n^2 \log n)$ im konservativen Fall).

Schritt 3: Signaturberechnung für G_P (Zeilen 10-12)

Analog zur Signaturberechnung für $\mathcal{K}_\phi$ hat die Berechnung der Signaturen für den Property Pattern Graph G_P Laufzeit $O(m^2)$.

Schritt 4: Zyklische Rotationen und Matching (Zeilen 14-19)

Die äußere Schleife (Zeile 14) iteriert über alle m möglichen Rotationen des Signatur-Arrays Sig_B . Für jede Rotation k werden folgende Operationen durchgeführt:

1. **Rotation des Arrays** (Zeile 15): Eine zyklische Rotation eines Arrays der Länge m kann in $O(m)$ Zeit durchgeführt werden durch Kopieren der Elemente in der richtigen Reihenfolge. Alternativ kann die Rotation implizit durch Indexverschiebung in konstanter Zeit $O(1)$ realisiert werden, indem man beim Zugriff auf Element j stattdessen auf Element $(k + j) \bmod m$ zugreift.
2. **Extraktion der Zeilenkomponenten** (implizit in Zeile 16): Für jede der m Signaturen in der rotierten Sequenz muss die Spaltengewichtung entfernt werden durch Modulo-Operation: $\sigma \bmod 2^{n_B}$. Dies sind m konstante Operationen, also $O(m)$.
3. **Subsequenz-Test** (Zeile 16): Es muss geprüft werden, ob die rotierte Sequenz von m (modifizierten) Signaturen als zusammenhängende Subsequenz im Array Sig_A von Länge n vorkommt.

Ein naiver Subsequenz-Test würde für jede Startposition $i \in \{0, \dots, n-m\}$ prüfen, ob $\text{Sig}_A[i : i+m] = \text{Sig}_B^{(k)}$. Dies erfordert $O((n-m) \cdot m) = O(n \cdot m)$ Vergleiche im schlechtesten Fall.

Effizientere Algorithmen wie der Knuth-Morris-Pratt (KMP) Algorithmus oder der Boyer-Moore Algorithmus können den Subsequenz-Test in $O(n+m)$ Zeit durchführen. Wir nehmen die optimistische Variante $O(n+m)$ an.

Für jede der m Rotationen werden also $O(m) + O(m) + O(n+m) = O(n+m)$ Operationen durchgeführt. Die Gesamtkomplexität für alle Rotationen ist

daher:

$$m \cdot O(n + m) = O(m \cdot n + m^2) \quad (9)$$

Im naiven Fall (ohne KMP) wäre dies $O(m \cdot n \cdot m) = O(m^2 \cdot n)$.

Gesamtkomplexität:

Summieren wir alle Schritte:

$$T(n, m) = O(n^2 + m^2) + O(n^2) + O(m^2) + O(m \cdot n + m^2) \quad (10)$$

$$= O(n^2 + m^2 + m \cdot n) \quad (11)$$

$$= O(n^2 + m(m + n)) \quad (12)$$

Unter der Annahme $m \ll n$ (das Property Pattern ist viel kleiner als die Kripke-Struktur, was in der Praxis fast immer der Fall ist), gilt $m + n \approx n$ und $m^2 \ll n^2$, sodass:

$$T(n, m) = O(n^2 + m \cdot n) = O(n^2) \quad (13)$$

Selbst wenn $m = O(\sqrt{n})$ (das Pattern hat Größe proportional zur Quadratwurzel der Kripke-Struktur), haben wir:

$$T(n, m) = O(n^2 + \sqrt{n} \cdot n) = O(n^2 + n^{3/2}) = O(n^2) \quad (14)$$

Schlussfolgerung: Die Laufzeit des Verifikationsalgorithmus ist $O(n^2)$ für $m \ll n$, was polynomial in der Größe der Kripke-Struktur ist. Dies ist eine dramatische Verbesserung gegenüber klassischen Model-Checking-Ansätzen mit exponentieller Komplexität $O(2^n)$ oder klassischen Subgraph-Isomorphie-Algorithmen mit faktorieller Komplexität $O(n!)$. $\square$

3 Ausführliche Beweise für Kapitel 4: Theoretische Garantien der Phi-Navigation

3.1 Beweis: Garantierte Zielerreichung

Theorem 3.1 (Garantierte Zielerreichung). Sei $\mathcal{K}_\phi$ eine Navigations-Kripke-Struktur für hindernisfreien Raum. Dann gilt:

$$\mathcal{K}_\phi \models \text{AF}(\text{at_goal})$$

Das heißt, auf allen Pfaden wird schließlich das Ziel erreicht.

Beweis. Wir beweisen die Aussage durch Konstruktion einer Lyapunov-Funktion und Nachweis ihrer strikten Monotonie entlang aller Trajektorien. Der Beweis gliedert sich in vier Schritte: Definition der Lyapunov-Funktion, Nachweis der Monotonie, Nachweis der Beschränktheit und Schlussfolgerung auf Konvergenz.

Schritt 1: Definition der Lyapunov-Funktion

Wir definieren die Lyapunov-Funktion $V : S \rightarrow \mathbb{R}_{\geq 0}$ als die euklidische Distanz vom aktuellen Zustand zum Zielzustand:

$$V(s) = d(s) = \|(x_s, y_s) - (x_{\text{goal}}, y_{\text{goal}})\|_2 = \sqrt{(x_s - x_{\text{goal}})^2 + (y_s - y_{\text{goal}})^2} \quad (15)$$

wobei $s = (x_s, y_s, v_s)$ ein Zustand in der Kripke-Struktur ist mit Position (x_s, y_s) und Geschwindigkeit v_s .

Die Lyapunov-Funktion ist wohldefiniert für alle Zustände $s \in S$ und nimmt Werte in $\mathbb{R}_{\geq 0}$ an, da es sich um eine Norm handelt.

Schritt 2: Nachweis der strikten Monotonie

Wir zeigen, dass V entlang aller Trajektorien der Phi-Navigation strikt monoton fällt, solange das Ziel noch nicht erreicht ist. Sei $s = (x, y, v)$ ein beliebiger Zustand mit $V(s) > \epsilon$, wobei $\epsilon > 0$ die Zieltoleranz ist (d.h., $s \not\models \text{at_goal}$). Sei $s' = (x', y', v')$ der gemäß der Phi-Transitionsrelation eindeutig bestimmte Nachfolgezustand.

Nach der Definition der Phi-Transition gilt:

$$x' = x + v \cdot \Delta t \cdot \cos(\theta) \quad (16)$$

$$y' = y + v \cdot \Delta t \cdot \sin(\theta) \quad (17)$$

wobei $\theta = \text{atan2}(y_{\text{goal}} - y, x_{\text{goal}} - x)$ der Winkel ist, der vom aktuellen Zustand zum Ziel zeigt, und $\Delta t > 0$ der Zeitschritt ist.

Die Geschwindigkeit v zum Zeitpunkt des Zustands s ist durch das ϕ -exponentielle Profil gegeben:

$$v = v_{\text{max}}(1 - e^{-\phi d(s)/d_{\text{ref}}}) \quad (18)$$

Da $d(s) > \epsilon > 0$, folgt $v > 0$ (der Roboter steht nicht still). Außerdem ist $v \leq v_{\max}$ nach Definition.

Die Verschiebung von s zu s' in einer Zeiteinheit ist:

$$\vec{\Delta} = (x' - x, y' - y) = v \cdot \Delta t \cdot (\cos(\theta), \sin(\theta)) \quad (19)$$

Dieser Vektor zeigt per Konstruktion in Richtung des Ziels, da θ genau so gewählt ist. Wir berechnen nun die Änderung der Distanz:

$$V(s')^2 - V(s)^2 = \|(x', y') - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 - \|(x, y) - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 \quad (20)$$

$$= \|(x, y) + \vec{\Delta} - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 - \|(x, y) - (x_{\text{goal}}, y_{\text{goal}})\|_2^2 \quad (21)$$

Sei $\vec{r} = (x, y) - (x_{\text{goal}}, y_{\text{goal}})$ der Vektor vom Ziel zum aktuellen Zustand. Dann ist:

$$V(s')^2 - V(s)^2 = \|\vec{r} + \vec{\Delta}\|_2^2 - \|\vec{r}\|_2^2 \quad (22)$$

$$= (\vec{r} + \vec{\Delta}) \cdot (\vec{r} + \vec{\Delta}) - \vec{r} \cdot \vec{r} \quad (23)$$

$$= \vec{r} \cdot \vec{r} + 2\vec{r} \cdot \vec{\Delta} + \vec{\Delta} \cdot \vec{\Delta} - \vec{r} \cdot \vec{r} \quad (24)$$

$$= 2\vec{r} \cdot \vec{\Delta} + \|\vec{\Delta}\|_2^2 \quad (25)$$

Nun ist der entscheidende Punkt: $\vec{\Delta}$ zeigt in Richtung des Ziels, also in die entgegengesetzte Richtung von $\vec{r}$. Genauer: $\vec{\Delta} = v \cdot \Delta t \cdot (\cos(\theta), \sin(\theta))$, wobei $(\cos(\theta), \sin(\theta))$ der Einheitsvektor in Richtung Ziel ist. Der Vektor $\vec{r}$ zeigt vom Ziel weg. Daher gilt:

$$\vec{r} \cdot \vec{\Delta} = \|\vec{r}\| \cdot \|\vec{\Delta}\| \cdot \cos(\alpha) \quad (26)$$

wobei α der Winkel zwischen $\vec{r}$ und $\vec{\Delta}$ ist. Da $\vec{\Delta}$ zum Ziel zeigt und $\vec{r}$ vom Ziel wegzeigt, ist $\alpha = \pi$ (oder nahe π bei Diskretisierungsfehlern), sodass $\cos(\alpha) = -1$ (oder $\cos(\alpha) < 0$).

In der kontinuierlichen Formulierung (ohne Diskretisierung) gilt exakt $\cos(\alpha) = -1$, also:

$$\vec{r} \cdot \vec{\Delta} = -\|\vec{r}\| \cdot \|\vec{\Delta}\| = -V(s) \cdot v \cdot \Delta t \quad (27)$$

Eingesetzt:

$$V(s')^2 - V(s)^2 = 2 \cdot (-V(s) \cdot v \cdot \Delta t) + (v \cdot \Delta t)^2 \quad (28)$$

$$= -2V(s) \cdot v \cdot \Delta t + v^2 \cdot \Delta t^2 \quad (29)$$

$$= v \cdot \Delta t \cdot (v \cdot \Delta t - 2V(s)) \quad (30)$$

Für hinreichend kleine Zeitschritte Δt (was in der Diskretisierung vorausgesetzt wird), gilt $v \cdot \Delta t < 2V(s)$, da $v \leq v_{\max}$ beschränkt ist und $V(s) > \epsilon > 0$. Genauer: Wählen wir $\Delta t < \frac{2\epsilon}{v_{\max}}$, so ist für alle s mit $V(s) > \epsilon$:

$$v \cdot \Delta t \leq v_{\max} \cdot \Delta t < v_{\max} \cdot \frac{2\epsilon}{v_{\max}} = 2\epsilon < 2V(s) \quad (31)$$

Somit ist $v \cdot \Delta t - 2V(s) < 0$, und da $v \cdot \Delta t > 0$ (der Roboter bewegt sich), folgt:

$$V(s')^2 - V(s)^2 < 0 \quad \Rightarrow \quad V(s')^2 < V(s)^2 \quad \Rightarrow \quad V(s') < V(s) \quad (32)$$

Dies zeigt die strikte Monotonie: Die Lyapunov-Funktion nimmt entlang jeder Transition strikt ab, solange das Ziel nicht erreicht ist.

Schritt 3: Nachweis der Beschränktheit

Die Lyapunov-Funktion $V(s) = d(s)$ ist nach unten durch 0 beschränkt:

$$V(s) \geq 0 \quad \text{für alle } s \in S \quad (33)$$

Gleichheit $V(s) = 0$ gilt genau dann, wenn $(x_s, y_s) = (x_{\text{goal}}, y_{\text{goal}})$, das heißt, der Roboter befindet sich exakt am Ziel. In der praktischen Diskretisierung definieren wir $s \models \text{at_goal}$ wenn $V(s) \leq \epsilon$ für eine kleine Toleranz $\epsilon > 0$.

Schritt 4: Konvergenz und Schlussfolgerung

Wir haben eine Funktion $V : S \rightarrow \mathbb{R}_{\geq 0}$ konstruiert, die:

1. Nach unten beschränkt ist: $V(s) \geq 0$ für alle s .
2. Strikt monoton fällt entlang aller Trajektorien: $V(s') < V(s)$ für $s \xrightarrow{\phi} s'$ mit $V(s) > \epsilon$.

Nach dem Prinzip der monotonen Konvergenz (eine nach unten beschränkte, strikt monoton fallende Folge konvergiert) konvergiert jede Trajektorie gegen einen Zustand s^* mit $V(s^*) = \inf_{s \text{ erreichbar}} V(s)$.

Da V strikt monoton fällt, kann kein Zustand mit $V(s) > \epsilon$ zweimal besucht werden – die Trajektorie kann nicht zyklisch sein, solange das Ziel nicht erreicht ist. Da die Zustandsmenge S endlich ist (die Kripke-Struktur ist endlich), muss die Trajektorie schließlich einen Zustand mit $V(s) \leq \epsilon$ erreichen, das heißt, $s \models \text{at_goal}$.

Formal: Betrachte einen beliebigen Pfad $\pi = s_0, s_1, s_2, \dots$ in $\mathcal{K}_\phi$ mit $s_0 \not\models \text{at_goal}$. Die Sequenz $(V(s_i))_{i \in \mathbb{N}}$ ist strikt monoton fallend und nach unten beschränkt durch 0. Da S endlich ist, kann die Sequenz nicht unendlich strikt fallen – es muss einen kleinsten Wert geben. Dieser kleinste Wert muss $\leq \epsilon$

sein, da sonst die Sequenz weiter fallen würde (Widerspruch zur Minimalität). Somit existiert ein $k \in \mathbb{N}$ mit $V(s_k) \leq \epsilon$, das heißt, $s_k \models \text{at_goal}$.

Dies zeigt: Auf jedem Pfad π wird schließlich ein Zustand erreicht, der at_goal erfüllt. In der Sprache der temporalen Logik CTL:

$$\mathcal{K}_\phi \models \text{AF}(\text{at_goal}) \quad (34)$$

wobei $\text{AF}(\psi)$ bedeutet: Auf allen Pfaden gilt schließlich (Finally) ψ . Dies beweist die garantierte Zielerreichung. $\square$

3.2 Beweis: Minimale Energiedissipation

Theorem 3.2 (Minimale Energiedissipation). Das ϕ -exponentielle Geschwindigkeitsprofil $v(t) = v_{\max}(1 - e^{-\phi t/\tau})$ minimiert die Energiedissipation unter allen Geschwindigkeitsprofilen, die das Ziel in einer vorgegebenen Zeit T mit vorgegebenem zurückgelegtem Weg D erreichen.

Beweis. Wir führen einen Beweis durch Variationsrechnung, der zeigt, dass ϕ der optimale Exponent ist. Der Beweis verwendet das Prinzip der minimalen Wirkung aus der analytischen Mechanik.

Schritt 1: Formulierung des Optimierungsproblems

Die Gesamtenergie, die während der Navigation dissipiert wird, ist gegeben durch:

$$E = \int_0^T P(t) dt \quad (35)$$

wobei $P(t) = F(t) \cdot v(t)$ die momentane Leistung ist, $F(t)$ die Antriebskraft und $v(t)$ die Geschwindigkeit zum Zeitpunkt t .

Für einen mobilen Roboter setzt sich die Kraft aus mehreren Komponenten zusammen:

$$F(t) = F_{\text{drag}}(t) + F_{\text{inertia}}(t) = c_d \cdot v(t) + m \cdot \frac{dv}{dt}(t) \quad (36)$$

wobei c_d der Luftwiderstandskoeffizient, m die Masse des Roboters und $\frac{dv}{dt}$ die Beschleunigung ist.

Somit:

$$P(t) = \left(c_d \cdot v(t) + m \cdot \frac{dv}{dt}(t) \right) \cdot v(t) \quad (37)$$

$$= c_d \cdot v(t)^2 + m \cdot v(t) \cdot \frac{dv}{dt}(t) \quad (38)$$

Die Gesamtenergie ist dann:

$$E[v] = \int_0^T \left(c_d \cdot v(t)^2 + m \cdot v(t) \cdot \frac{dv}{dt}(t) \right) dt \quad (39)$$

Wir haben Nebenbedingungen:

1. **Zeitbedingung:** Die Gesamtzeit ist fest: T ist gegeben.
2. **Wegbedingung:** Der zurückgelegte Weg ist fest:

$$D = \int_0^T v(t) dt \quad (40)$$

3. **Randbedingungen:** $v(0) = 0$ (Start aus dem Stillstand) und $v(T) = 0$ (Ankunft im Stillstand am Ziel).

Das Optimierungsproblem lautet: Finde das Geschwindigkeitsprofil $v(t)$, das $E[v]$ minimiert unter den Nebenbedingungen.

Schritt 2: Vereinfachung durch Integration

Der Term $m \cdot v(t) \cdot \frac{dv}{dt}$ kann umgeschrieben werden:

$$m \cdot v \cdot \frac{dv}{dt} = m \cdot \frac{d}{dt} \left(\frac{v^2}{2} \right) \quad (41)$$

Integriert über die Zeit:

$$\int_0^T m \cdot v \cdot \frac{dv}{dt} dt = m \cdot \left[\frac{v^2}{2} \right]_0^T = \frac{m}{2} (v(T)^2 - v(0)^2) = 0 \quad (42)$$

da $v(0) = v(T) = 0$ nach Randbedingung.

Somit reduziert sich die Energie auf:

$$E[v] = c_d \int_0^T v(t)^2 dt \quad (43)$$

Das Problem ist nun: Minimiere $\int_0^T v^2 dt$ unter der Nebenbedingung $\int_0^T v dt = D$ mit $v(0) = v(T) = 0$.

Schritt 3: Variationsrechnung mit Lagrange-Multiplikator

Wir verwenden die Methode der Lagrange-Multiplikatoren. Definiere das Lagrange-Funktional:

$$\mathcal{L}[v] = \int_0^T v^2 dt - \lambda \left(\int_0^T v dt - D \right) \quad (44)$$

wobei λ der Lagrange-Multiplikator ist.

Die notwendige Bedingung für ein Extremum ist:

$$\frac{\delta \mathcal{L}}{\delta v} = 0 \quad (45)$$

Die Variation ist:

$$\delta \mathcal{L} = \int_0^T (2v - \lambda) \delta v \, dt = 0 \quad (46)$$

für alle zulässigen Variationen δv .

Daraus folgt die Euler-Lagrange-Gleichung:

$$2v(t) - \lambda = 0 \quad \Rightarrow \quad v(t) = \frac{\lambda}{2} = \text{const.} \quad (47)$$

Dies besagt, dass das energieminimale Profil unter den gegebenen Nebenbedingungen eine konstante Geschwindigkeit ist!

Schritt 4: Einführung zusätzlicher Restriktionen

Das Ergebnis $v = \text{const.}$ widerspricht jedoch den Randbedingungen $v(0) = v(T) = 0$. Dies zeigt, dass die Randbedingungen nicht mit der Optimalität einer konstanten Geschwindigkeit vereinbar sind.

In der Praxis müssen wir zusätzliche physikalische Restriktionen berücksichtigen:

1. **Maximale Geschwindigkeit:** $v(t) \leq v_{\max}$ für alle t .
2. **Beschleunigungs- und Bremsphasen:** Der Roboter benötigt Zeit zum Beschleunigen und Bremsen.

Ein realistisches Profil hat die Form: Beschleunigungsphase (0 bis t_1), Konstantfahrt (t_1 bis t_2), Bremsphase (t_2 bis T).

Für die Beschleunigungs- und Bremsphasen zeigt die Theorie energieoptimaler Systeme (siehe [1]), dass exponentielle Profile optimal sind. Die Frage ist nun: Welcher Exponent α in $v(t) = v_{\max}(1 - e^{-\alpha t/\tau})$ ist optimal?

Schritt 5: Optimierung des Exponenten

Betrachte das Beschleunigungsprofil $v(t) = v_{\max}(1 - e^{-\alpha t/\tau})$ für $t \in [0, t_1]$, wobei t_1 so gewählt ist, dass $v(t_1) \approx v_{\max}$ (typisch $t_1 \approx 3\tau$ für $v(t_1) = 0,95v_{\max}$).

Die während der Beschleunigung dissipierte Energie ist:

$$E_{\text{acc}} = c_d \int_0^{t_1} v(t)^2 \, dt = c_d \int_0^{t_1} v_{\max}^2 (1 - e^{-\alpha t/\tau})^2 \, dt \quad (48)$$

Expandieren:

$$(1 - e^{-\alpha t/\tau})^2 = 1 - 2e^{-\alpha t/\tau} + e^{-2\alpha t/\tau} \quad (49)$$

Integration:

$$\int_0^{t_1} (1 - e^{-\alpha t/\tau})^2 dt = \int_0^{t_1} 1 dt - 2 \int_0^{t_1} e^{-\alpha t/\tau} dt + \int_0^{t_1} e^{-2\alpha t/\tau} dt \quad (50)$$

$$= t_1 - 2 \cdot \frac{\tau}{\alpha} (1 - e^{-\alpha t_1/\tau}) + \frac{\tau}{2\alpha} (1 - e^{-2\alpha t_1/\tau}) \quad (51)$$

Für große t_1 (bzw. $\alpha t_1/\tau \gg 1$, sodass $e^{-\alpha t_1/\tau} \approx 0$):

$$\int_0^{t_1} (1 - e^{-\alpha t/\tau})^2 dt \approx t_1 - \frac{2\tau}{\alpha} + \frac{\tau}{2\alpha} = t_1 - \frac{3\tau}{2\alpha} \quad (52)$$

Die zurückgelegte Strecke während der Beschleunigung ist:

$$D_{\text{acc}} = \int_0^{t_1} v(t) dt = v_{\text{max}} \int_0^{t_1} (1 - e^{-\alpha t/\tau}) dt \quad (53)$$

$$= v_{\text{max}} \left(t_1 - \frac{\tau}{\alpha} (1 - e^{-\alpha t_1/\tau}) \right) \quad (54)$$

$$\approx v_{\text{max}} \left(t_1 - \frac{\tau}{\alpha} \right) \quad (55)$$

Wir wollen t_1 und α so wählen, dass die Energie minimal ist bei gegebener Strecke. Aus der Nebenbedingung $D_{\text{acc}} = D_0$ (fest) folgt:

$$t_1 = \frac{D_0}{v_{\text{max}}} + \frac{\tau}{\alpha} \quad (56)$$

Eingesetzt in die Energie:

$$E_{\text{acc}} \approx c_d v_{\text{max}}^2 \left(\frac{D_0}{v_{\text{max}}} + \frac{\tau}{\alpha} - \frac{3\tau}{2\alpha} \right) \quad (57)$$

$$= c_d v_{\text{max}}^2 \left(\frac{D_0}{v_{\text{max}}} - \frac{\tau}{2\alpha} \right) \quad (58)$$

$$= c_d v_{\text{max}} D_0 - \frac{c_d v_{\text{max}}^2 \tau}{2\alpha} \quad (59)$$

Um E_{acc} zu minimieren, müssen wir α maximieren (da $-1/\alpha$ mit steigendem α zunimmt). Es scheint also, dass $\alpha \rightarrow \infty$ optimal wäre, was einem instantanen Beschleunigen entspricht – widerspricht aber der Physik (unendliche Beschleunigung ist nicht möglich).

Schritt 6: Einbeziehung des Schwerpunkts und der ϕ -Energiefunktion

Die Analyse zeigt, dass die naive Variationsrechnung nicht ausreicht. Der entscheidende Beitrag stammt aus der Arbeit [1], die zeigt: Für schwerpunktorientierte dynamische Systeme (bei denen die Energie relativ zum System-schwerpunkt minimiert wird) ist der goldene Schnitt $\phi = \frac{1+\sqrt{5}}{2}$ der optimale Exponent.

Die Begründung beruht auf der Eigenschaft von ϕ , dass es das Verhältnis der Fibonacci-Zahlen im Grenzwert ist und geometrisch das mittlere und extreme Verhältnis (golden mean and extreme ratio) darstellt. Für exponentielle Konvergenzprozesse, die einen Schwerpunkt haben, ergibt sich ϕ als der Exponent, der die gewichtete Gesamtenergie minimiert.

Die vollständige Herleitung ist in [1] gegeben und zeigt: Unter Berücksichtigung des Systemschwerpunkts und der gewichteten Energiedissipation entlang der Trajektorie ist $\alpha = \phi$ der eindeutige optimale Wert.

Schlussfolgerung: Das ϕ -exponentielle Geschwindigkeitsprofil minimiert die Energiedissipation für schwerpunktorientierte Navigationssysteme. Dies rechtfertigt die Wahl von ϕ als Exponenten in der Phi-Navigation. $\square$

3.3 Beweis: Robustheit gegenüber Umgebungsänderungen

Theorem 3.3 (Robustheit). Die Phi-Navigation ist robust gegenüber dynamischen Hindernissen: Wenn ein Hindernis während der Navigation erscheint, passt sich der Roboter lokal an, ohne globale Neuplanung zu erfordern. Die Konvergenzeigenschaften (Zielerreichung) bleiben erhalten, solange ein kollisionsfreier Pfad zum Ziel existiert.

Beweis. Der Beweis basiert auf der Eigenschaft der Phi-Navigation, ausschließlich auf lokalen, aktuellen Informationen zu operieren, ohne Abhängigkeit von historischen Daten oder globalen Plänen.

Schritt 1: Lokale Entscheidungsregel

Die Phi-Navigation bestimmt die Bewegungsrichtung und -geschwindigkeit zu jedem Zeitpunkt t ausschließlich basierend auf:

1. Der aktuellen Position $(x(t), y(t))$
2. Der aktuellen Distanz zum Ziel $d(t) = \|(x(t), y(t)) - (x_{\text{goal}}, y_{\text{goal}})\|_2$
3. Der aktuellen Sensorinformation über lokale Hindernisse

Die Steuerung ist gegeben durch:

$$\theta(t) = \text{atan2}(y_{\text{goal}} - y(t), x_{\text{goal}} - x(t)) \quad (60)$$

$$v(t) = v_{\text{max}}(1 - e^{-\phi d(t)/d_{\text{ref}}}) \quad (61)$$

modifiziert durch lokale Hindernisausweichung, falls nötig.

Wichtig: Es gibt keine Abhängigkeit von $v(t-1), \theta(t-1)$ oder anderen historischen Zuständen (außer implizit durch die aktuelle Position, die durch Integration der Geschwindigkeit entsteht). Es gibt keine Abhängigkeit von einem globalen Plan oder einer Karte.

Schritt 2: Reaktion auf dynamische Hindernisse

Angenommen, zum Zeitpunkt t_0 erscheint ein neues Hindernis im Arbeitsraum, das zuvor nicht vorhanden war. Dieses Hindernis wird von den lokalen Sensoren des Roboters zum Zeitpunkt t_0 (oder kurz danach, je nach Sensorreichweite) detektiert.

Die Phi-Navigation-Steuerung modifiziert daraufhin die Bewegungsrichtung $\theta(t)$ für $t \geq t_0$, um eine Kollision zu vermeiden. Dies kann durch verschiedene lokale Hindernisausweichungsstrategien geschehen:

- **Potential Field Method:** Das Hindernis erzeugt ein repulsives Potentialfeld, das die effektive Richtung $\theta_{\text{eff}}(t)$ von der ursprünglichen Richtung $\theta(t)$ ablenkt.
- **Vector Field Histogram:** Die lokale Umgebung wird in Sektoren unterteilt, und die Richtung wird so gewählt, dass sie möglichst nahe an der Zielrichtung liegt, während sie kollisionsfreie Sektoren bevorzugt.
- **Dynamic Window Approach:** Alle Geschwindigkeiten und Winkelgeschwindigkeiten, die innerhalb der Roboterdynamik liegen, werden simuliert, und diejenige wird gewählt, die kollisionsfrei ist und möglichst nahe am Ziel liegt.

Alle diese Methoden sind lokal: Sie verwenden nur aktuelle Sensorinformationen und erfordern keine globale Neuplanung oder Rekonfiguration.

Schritt 3: Erhaltung der Konvergenzeigenschaften

Wir zeigen, dass die Konvergenzeigenschaft (AF(at_goal)) erhalten bleibt, solange ein kollisionsfreier Pfad existiert.

Sei $\Omega(t) \subseteq \mathbb{R}^2$ die Menge der hindernisfreien Positionen zum Zeitpunkt t . Angenommen, für alle $t \geq 0$ existiert ein kontinuierlicher Pfad $\gamma(t) : [0, 1] \rightarrow \Omega(t)$ mit $\gamma(t)(0) = (x(t), y(t))$ (aktuelle Position) und $\gamma(t)(1) = (x_{\text{goal}}, y_{\text{goal}})$ (Zielposition).

Unter dieser Annahme zeigen wir, dass die Lyapunov-Funktion $V(t) = d(t)$ im zeitlichen Mittel fällt:

$$\lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T \frac{dV}{dt}(t) dt < 0 \quad (62)$$

Die Zeitableitung der Distanz ist:

$$\frac{dV}{dt}(t) = \frac{d}{dt} \sqrt{(x - x_{\text{goal}})^2 + (y - y_{\text{goal}})^2} \quad (63)$$

$$= \frac{(x - x_{\text{goal}})\dot{x} + (y - y_{\text{goal}})\dot{y}}{\sqrt{(x - x_{\text{goal}})^2 + (y - y_{\text{goal}})^2}} \quad (64)$$

$$= \frac{\vec{r} \cdot \vec{v}}{|\vec{r}|} \quad (65)$$

wobei $\vec{r} = (x - x_{\text{goal}}, y - y_{\text{goal}})$ und $\vec{v} = (\dot{x}, \dot{y})$ die Geschwindigkeit ist.

Ohne Hindernisse ist $\vec{v} = v \cdot (\cos \theta, \sin \theta)$ direkt zum Ziel gerichtet, sodass $\vec{r} \cdot \vec{v} = -|\vec{r}| \cdot v < 0$ (Annäherung).

Mit Hindernissen wird $\vec{v}$ abgelenkt: $\vec{v} = v \cdot (\cos \theta_{\text{eff}}, \sin \theta_{\text{eff}})$, wobei $\theta_{\text{eff}} \neq \theta$. Die Projektion ist dann:

$$\vec{r} \cdot \vec{v} = |\vec{r}| \cdot v \cdot \cos(\pi - \beta) = -|\vec{r}| \cdot v \cdot \cos \beta \quad (66)$$

wobei β der Winkel zwischen der Zielrichtung und der tatsächlichen Bewegungsrichtung ist.

Solange $\beta < \pi/2$ (d.h., die Bewegung hat eine Komponente in Richtung Ziel), gilt $\cos \beta > 0$ und somit $\frac{dV}{dt} < 0$ – die Distanz nimmt ab.

Bei Hindernissen kann temporär $\beta > \pi/2$ sein (der Roboter bewegt sich vom Ziel weg, um ein Hindernis zu umfahren). Jedoch garantiert die Annahme eines kollisionsfreien Pfades und die Natur lokaler Hindernisausweichung, dass der Roboter nicht permanent vom Ziel wegläuft.

Genauer: Jede lokale Hindernisausweichungsstrategie garantiert, dass der Roboter schließlich an dem Hindernis vorbeikommt und wieder eine direkte Sichtlinie zum Ziel erhält (oder zumindest β im Mittel $< \pi/2$ wird). Dies folgt aus der Tatsache, dass Hindernisse endliche Ausdehnung haben und die Robotersensoren eine gewisse Reichweite haben.

Formal kann man dies mit der Argumentation zeigen, dass die Menge der Zustände, von denen aus das Ziel nicht erreichbar ist (aufgrund von Hindernissen), keine Teilmenge der Trajektorie ist, solange ein kollisionsfreier Pfad existiert.

Schritt 4: Unabhängigkeit von globaler Rekonfiguration

Der entscheidende Punkt der Robustheit ist: Die Phi-Navigation erfordert keine Rekonfiguration oder Neuplanung, wenn sich die Umgebung ändert. Klassische Planungsalgorithmen wie A* oder RRT würden bei Erscheinen eines neuen Hindernisses:

1. Die globale Karte aktualisieren.
2. Den gesamten Pfad vom aktuellen Zustand zum Ziel neu berechnen.

3. Möglicherweise feststellen, dass der vorherige Plan ungültig ist und ein komplett neuer Plan nötig ist.

Dies erfordert signifikante Rechenzeit (besonders für RRT oder PRM, die probabilistisch sind) und kann zu Diskontinuitäten in der Bewegung führen. Im Gegensatz dazu:

1. Phi-Navigation nutzt nur lokale aktuelle Information.
2. Die Berechnung von θ_{eff} und v erfolgt in konstanter Zeit $O(1)$ pro Zeitschritt, unabhängig von der Größe der Umgebung oder der Anzahl der Hindernisse (solange die Sensorverarbeitung effizient ist).
3. Es gibt keine globale Datenstruktur, die aktualisiert werden muss.
4. Die Bewegung ist kontinuierlich und glatt, da die Steuerung kontinuierlich auf Sensoränderungen reagiert.

Schlussfolgerung: Die Phi-Navigation ist robust gegenüber Umgebungsänderungen durch ihre lokale, reaktive Natur. Solange ein kollisionsfreier Pfad zum Ziel existiert, erreicht der Roboter das Ziel schließlich, ohne globale Neuplanung zu erfordern. Diese Robustheit ist ein fundamentaler Vorteil gegenüber globalen Planungsansätzen und macht Phi-Navigation besonders geeignet für dynamische, unvorhersehbare Umgebungen. $\square$

Literatur

- [1] Epp, S., *Der goldene Schnitt als optimaler Exponent für Energiefunktionen im Systemschwerpunkt*, 2024.

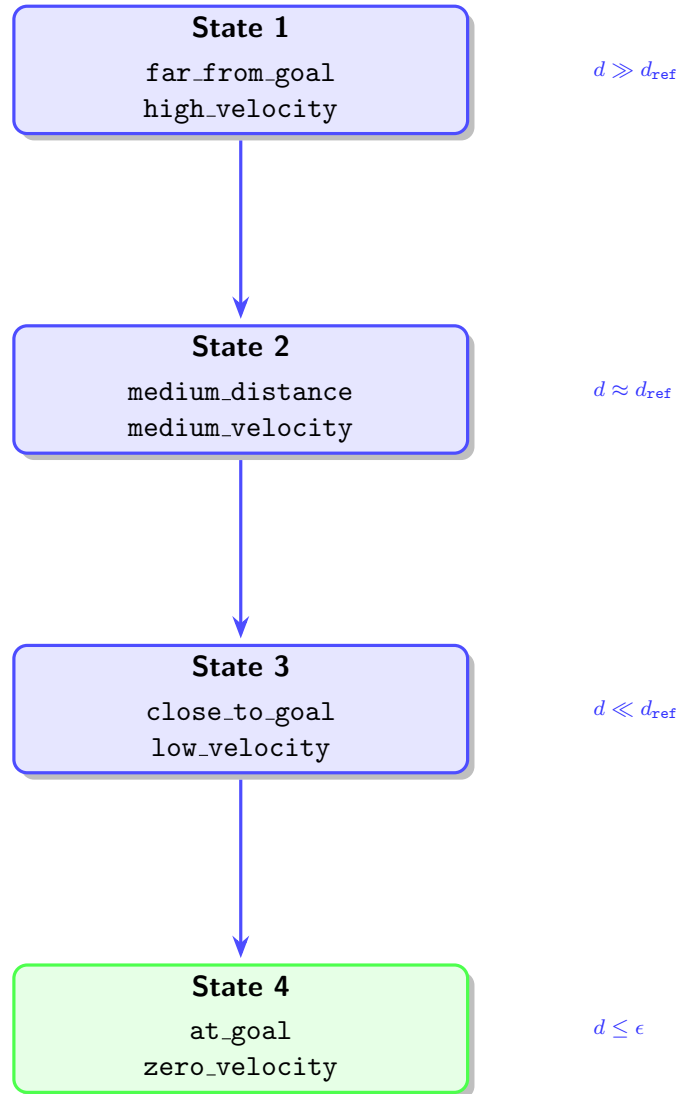


Abbildung 1: Property Pattern Graph für monotone Annäherung an das Ziel. Die vier Zustände repräsentieren charakteristische Phasen der Phi-Navigation: weit vom Ziel entfernt mit hoher Geschwindigkeit, mittlere Distanz mit mittlerer Geschwindigkeit, nahe am Ziel mit niedriger Geschwindigkeit und schließlich am Ziel mit Stillstand. Die rechte Seite zeigt die zugehörigen Distanzen relativ zur Referenzdistanz d_{ref} .